Concepte fundamentale ale limbajelor de programare

Structuri de control

Curs 12

conf. dr. ing. Ciprian-Bogdan Chirila

Universitatea Politehnica Timisoara
Departamentul de Calculatoare si Tehnologia Informatiei

10 decembrie 2024

Cuprins

1. Introducere
2. Structuri de control la nivel de instructiune
   - Secventa
   - Selectia
   - Repetitia
3. Subprograme
   - Efecte colaterale
   - Pseudonime
4. Exceptii
   - Exceptii in Ada
   - Exceptii in C#
   - Exceptii in Python
5. Pachete
   - Pachete in Java
   - Namespace-uri in C Sharp
6. Module
   - Module Java

Structuri de control

Ne concentram pe mecanisme care permit programatorului sa controleze fluxul de executie la:

- nivel de instructiune
- nivel de subprogram

Cuprins

1. Introducere
2. Structuri de control la nivel de instructiune
   - Secventa
   - Selectia
   - Repetitia
3. Subprograme
   - Efecte colaterale
   - Pseudonime
4. Exceptii
   - Exceptii in Ada
   - Exceptii in C#
   - Exceptii in Python
5. Pachete
   - Pachete in Java
   - Namespace-uri in C Sharp
6. Module
   - Module Java

Structuri de control la nivel de instructiune

- specifica ordinea in care instructiunile unui program individual sunt executate
- sunt grupate in trei categorii
  - Secventa
  - Selectia
  - Repetitia

Secventa

- este cea mai simpla structura de control
- este specifica limbajelor imperative
- nu ne referim la limbaje concurente
- instructiunile sunt executate in ordinea in care sunt scrise

Secventa

- in multe limbaje de programare seturi de instructiuni pot fi grupate impreuna pentru a forma o instructiune compusa
- in Python
  - sunt indentate cu un tab
- In C, Java, C#, JavaScript, TypeScript

```
{
  ...
}
```

Selectia

- ne permite sa selectam
  - o alternativa dintre doua sau mai multe disponibile
  - depinzand de o conditie logica
- in limbaje de tipul Algol-Pascal avem

```
if condition then
 sequence_of_instructions
else
 sequence_of_instructions
end if;
```

Selectia in Python

```
if test expression:
 statement(s)

if test expression:
 Body_of_if
else:
 Body_of_else

if test expression:
 Body_of_if
elif test expression:
 Body_of_elif
else:
 Body_of_else
```

Selectia intre alternative multiple

- case
  - like in Pascal, Ada, Algol 68
- switch
  - like in C, Java, C#, JavaScript
- selectia este bazata pe o valoare selectoare de tip scalar
- programatorul trebuie sa specifice variantele si valorile pentru care fiecare alternativa este selectata
- exista o optiune prin care se poate specifica o varianta ce va fi aleasa daca nu exista potrivire

Exemplu in Pascal

```
if mark <= 10 then
 case mark of
  1,2,3,4 : writeln('failed');
  5,6,7 : writeln('passed');
  8,9 : writeln('good');
  10 : writeln('excellent');
 end
else
 writeln('wrong mark');
```

Exemplu in Ada

```
case mark of
 when 1..4 => put_line("failed");
 when 5|6|7 => put_line("passed");
 when 8|9 => put_line("good");
 when 10 => put_line("excellent");
 when others => put_line("wrong mark");
end case;
```

Exemple in limbaje cu sintaxa C

```
switch(mark)
{
 case 1: case 2: case 3: case 4: printf("failed"); break;
 case 5: case 6: case 7: printf("passed"); break;
 case 8: case 9: printf("good"); break;
 case 10: printf("excellent"); break;
 default: printf("wrong mark"); break;
}
```

Inlocuitori in Python

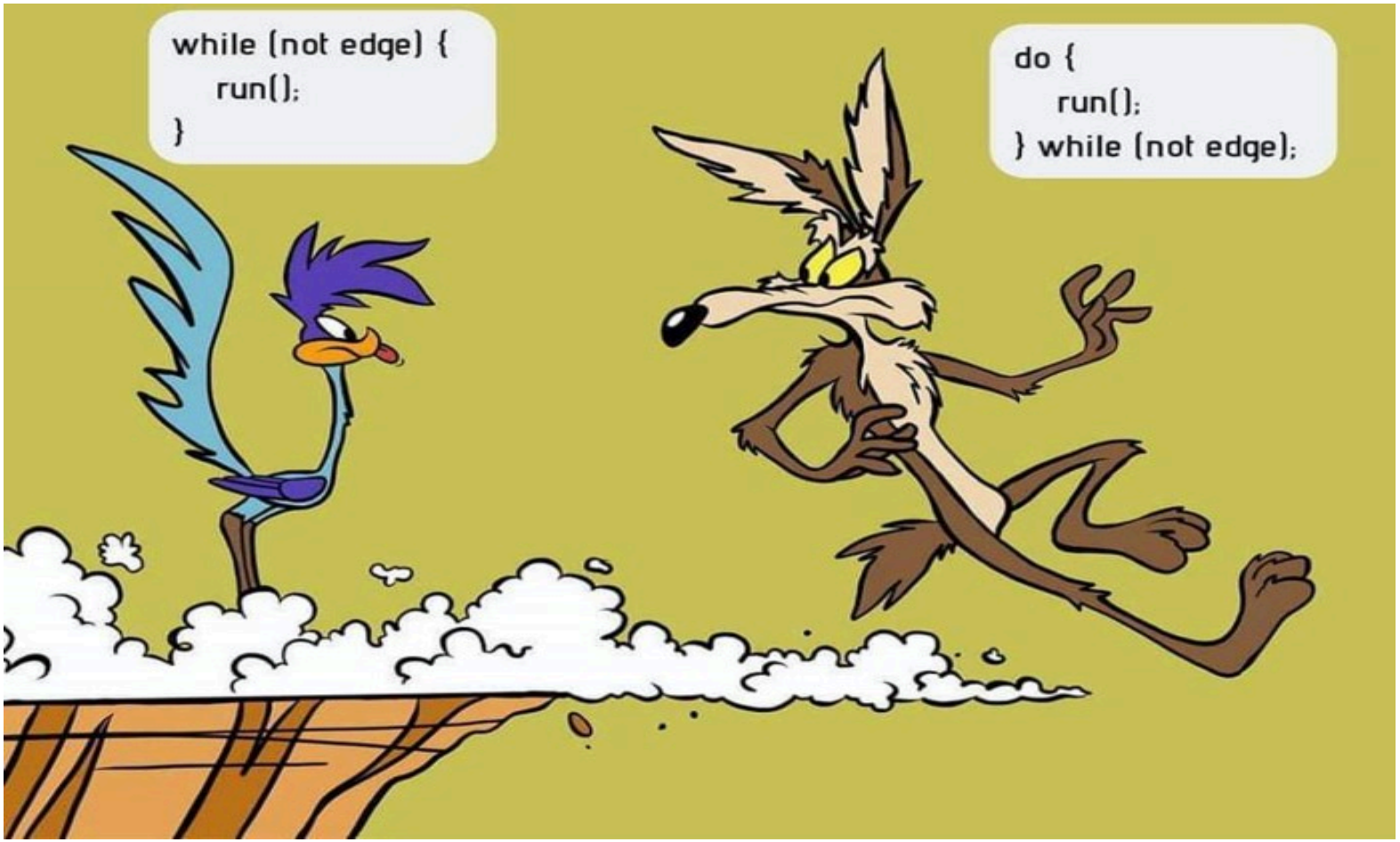
```
def numbers_to_strings(argument):
 switcher =
 {
  1:"failed",2:"failed",3:"failed",4:"failed",
  5:"passed",6:"passed",7:"passed",
  8:"good",9:"good",
  10:"excellent",
 }
 return switcher.get(argument, "wrong mark")

if __name__ == "__main__":
 argument=5
 print (numbers_to_strings(argument))
```

Repetitia

- este mecanismul de baza pentru a face calcule complexe
- inseamna sa executi in mod repetat o instructiunie sau un set de instructiuni
- structurile repetitive sunt controlate
  - de conditii
  - de contoare

Structuri repetitive controlate de conditii



Structuri repetitive controlate de conditii

- repetitii cu test initial
  - Pascal: while condition do
  - C, Java, C#, JavaScript, TypeScript:
    `while(condition) instruction;`
- repetitii cu test final
  - Pascal: repeat instr_sequence until condition
  - C, Java, C#, JavaScript, TypeScript:
    `do instruction while(condition);`

Structuri repetitive controlate de contoare

- contorul avanseaza
  - de la o valoare initiala
  - la o valoare finala
- forma generala este

```
for variable:= initial_value to final_value step step_value do
 instruction
```

Structuri repetitive controlate de contoare

- in PI/I and Algol 68

```
for
 variable from initial_value
 by step_value
 to final value
 while condition do
sequence_of_instructions
```

Structuri repetitive controlate de contoare

- in limbajele cu sintaxa C

```
for(expr1; expre2; expr3)
 instruction;

expr1;
while(expr2)
{
 instruction;
 expr3;
}
```

Iesirea din repetitie

- salt neconditionat
  - goto
- instructiuni de salt specializat pentru a iesi din repetitii
  - in Ada
    - exit
  - in limbaje cu sintaxa C
    - break
- doar o repetitie, dar nu si cele inconjuratoare
  - In Ada
    - exit name
    - "name" este eticheta repetitiei inconjuratoare
  - in limbajele cu sintaxa C
    - continue

Cuprins

1. Introducere
2. Structuri de control la nivel de instructiune
   - Secventa
   - Selectia
   - Repetitia
3. Subprograme
   - Efecte colaterale
   - Pseudonime
4. Exceptii
   - Exceptii in Ada
   - Exceptii in C#
   - Exceptii in Python
5. Pachete
   - Pachete in Java
   - Namespace-uri in C Sharp
6. Module
   - Module Java

Structuri de control la nivel de subprograme
Subprograms

- efecte colaterale
  - modificarile provocate de un sunprogram asupra unei entitati care nu este locala subprogramului
  - sunt problematice in special in cazul functiilor
  - `v=a+f(a,b)+c;`
- pseudonime
  - un obiect poate fi referit de doua sau mai multe nume
  - poate aparea in cazul mecanismului de transfer a parametrilor prin adresa

Exemple de pseudonime in C++

```
int y;
----------------
void p(int &x)
{
 x=2*x;
 y=x+y;
};
----------------
y=1;
p(y);
----------------
```

Exemple de pseudonime in C++

- cand doua sau mai multe argumente sunt transmise prin adresa reprezentand acelasi obiect

```
int z;
------------------
void p(int &x,int &y)
{
 x=2*x;
 y=2*y;
}
------------------
z=3;
p(z,z);
```

Exemple de pseudonime in C++

- cand o structura sau componente a acelei structuri sunt transmise ca parametri

```
typedef float tab_t[100];
tab_t tab;
procedure pp(int &x, tab_t &tab)
{
 ...
}
-------------------
pp(tab[3],tab);
-------------------
```

Pseudonime in tablouri

- cand doua argumente sunt doua elemente ale aceluiasi tablou
- `p(t[i],t[j]);`
- pseudonimele apar daca `i==j`

Cuprins

1. Introducere
2. Structuri de control la nivel de instructiune
   - Secventa
   - Selectia
   - Repetitia
3. Subprograme
   - Efecte colaterale
   - Pseudonime
4. Exceptii
   - Exceptii in Ada
   - Exceptii in C#
   - Exceptii in Python
5. Pachete
   - Pachete in Java
   - Namespace-uri in C Sharp
6. Module
   - Module Java

Tratarea exceptiilor in Ada

- 5 tipuri de exceptii
- constraint_error
  - violarea limitelor unui subdomeniu
  - referirea ilegala a unui cand dintr-un articol cu variante
  - referirea unui ponter null
- numeric_error
  - arithmetical overflow
- storage_error
  - depasirea spatiului de memorie
- select_error, tasking_error
  - erori legate de concurenta

Tratarea exceptiilor in Ada

- declararea exceptiilor
  - error, end : exception;
- ridicarea exceptiilor
  - raise error;

Exemplu de tratare a exceptiilor

```
function f(x : float) return float is
negative : exception;
begin
if x<0 then
 raise negative;
else
 return 1/sqrt(x);
end if;
exception
 when numeric_error =>
  return 0; -- return 0 if x is 0
 when negative =>
  return -1; -- return -1 if x is negative
end f;
```

Exemplu de tratare a exceptiilor

```
package stack is
 error: exception;
 type stack_type(max_no:integer) is limited private;
 function pop(s:in out stack_type) return integer;
 procedure push(s:in out stack_type; x: integer);
 function top(s:stack_type) return integer;
 procedure init(s:out stack_type);
private
 type stack_type(max_no:integer) is
 record
  tab_st:array(1..max_no) of integer;
  ind:integer;
  end record;
end stack;
```

Exemplu de tratare a exceptiilor

```
package body stack is

 function empty(s:stack_type) return boolean is
 begin ... end empty;

 function overflow(s:stack_type) return boolean is
 begin ... end overflow;

 function pop(s: in out stack_type) return integer is
 begin
  if not empty(s) then
   s.ind:=s.ind-1;
    return s.tab_st(s.ind);
   else raise error;
end if;
 end pop;
```

Exemplu de tratare a exceptiilor

```
procedure push(s: in out stack_type; x:integer) is
begin
 if not overflow(s) then
  s.tab_st(s.ind):=x;
  s.ind:=s.ind+1;
 else raise error;
 end if;
end push;
```

Exemplu de tratare a exceptiilor

```
function top(s:stack_type) return integer is
begin
 if not empty(s) then
  return s.tab_st(s.ind-1);
 else
  raise error;
 end if;
end top;

function init(s: out stack_type) is
begin
 ...
end init;

begin
end stack;
```

Exemplu de tratare a exceptiilor

```
procedure st is
use stack;
stk:stack_type(100);
-----------------------
init(stk);
-----------------------
push(stk,10);
-----------------------
i:=top(stk);
-----------------------
```

Exemplu de tratare a exceptiilor

```
exception
 when error =>
  put_line("error using stack");
  while not empty(stk) loop
   put(pop(stk));
  end loop;
end st;
```

Tratarea exceptiilor in C#

- sunt similare cu C++ sau Java
- exceptiile sunt reprezentate prin clase
- Class System.Exception
- toate clasele reprezentand exceptii trebuie sa fie derivate din clasa predefinita Exception care este parte a spatiului de nume System
- clase derivate din Exception sunt
  - SystemException
    - generate de motorul de executie
  - AplicationException
    - generate de aplicatiile de program
    - pot fi derivate de programator prin crearea propriilor exceptii

Clase de exceptii derivate din SystemException

- ArrayTypeMismatchException
  - tipul atribuirii este incompatibil cu tipul tabloului
- DivideByZeroException
  - tentativa de a imparti cu zero
- IndexOutOfRangeException
  - indexul iese din limitele tabloului
- InvalidCastException
  - operatie de cast incorecta la rulare

Clase de exceptii derivate din SystemException

- OutOfMemoryException
  - memorie insuficienta pentru operatorul new de alocare
- OverflowException
  - supradepasire aritmetica
- StackOverflowException
  - depasirea a capacitatii stivei

Elemente de baza in tratarea exceptiilor

- cuvinte cheie
  - try
  - catch
  - throw
  - finally

Functionarea exceptiilor

- try
  - blocul try contine instructiuni sensibile la erori ce trebuiesc a fi verificate
- throw
  - daca o exceptie are loc atunci ea este aruncata
- catch
  - programul poate intercepta aceasta exceptie si o trateaza in acord cu cerintele aplicatiei
- exceptiile sunt lansate
  - in mod automat de motorul de executie C#
  - manual, utilizand cuvantul cheie throw

Tratarea exceptiilor

- codul ce va fi executat cand se iese din bloc trebuie pus in blocul finally
- pentru a prinde orice exceptie
- the orice tip
- clauza catch se foloseste fara parametru
- astfel, este creata o rutina universala pentru a intercepta si trata toate exceptiile

Exemplu simplu de tratare a exceptiilor

```
using System;
class Example1
{
 public static void Main()
 {
  int [] nums = new int[4];
  try
  {
   Console.WriteLine("Before exception");
   // we generate an exception of invalid index
   nums[7] = 10;
   Console.WriteLine("Message not to be printed");
  }
```

Exemplu simplu de tratare a exceptiilor

```
  catch(IndexOutOfRangeException)
  {
   // we intercept the exception
   Console.WriteLine("Index out of bounds");
  }
  Console.WriteLine("After catch");
 }
}
```

Exemplu simplu de tratare a exceptiilor

- text afisat:

```
Before exception
Index out of bounds
After catch
```

Generarea manuala a unei exceptii

```
using System;
class Example2
{
 public static void Main()
 {
  try
  {
   Console.WriteLine("Before throw");
   // launching  exception
   throw new DivideByZeroException();
  }
  catch(DivideByZeroException)
  {
   // intercepting the exception
   Console.WriteLine("Exception intercepted");
  }
  Console.WriteLine("After try/catch");
}
```

Generarea manuala a unei exceptii

- text afisat

```
Before throw
Exception intercepted
After try/catch
```

Excepții în Python

Ce sunt excepțiile?

- Erori apărute în timpul execuției programului.
- Exemple: `ZeroDivisionError`, `ValueError`, `FileNotFoundError`.

Cum se tratează excepțiile?

- Folosind blocurile `try-except`.

Exemplu:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Împărțirea la zero nu este permisă!")
```

Blocul try-except-finally

- `try`: Se încearcă executarea unui cod care ar putea produce erori.
- `except`: Cod care gestionează eroarea specifică.
- `finally`: Cod care se execută întotdeauna, indiferent de erori.

Exemplu:

```
try:
    file = open("fisier.txt", "r")
    data = file.read()
except FileNotFoundError:
    print("Fișierul nu a fost găsit!")
finally:
    file.close()
```

Utilizarea contextului `with`

Ce este `with`?

- Oferă o metodă sigură și mai simplă pentru a lucra cu resurse externe.
- Resursele sunt închise automat la finalul blocului `with`.

Avantaje:

- Evită erorile cauzate de uitarea de a închide resursele (ex. fișiere, conexiuni).

Exemplu:

```
with open("fisier.txt", "r") as file:
    data = file.read()
    print(data)
    # Nu este nevoie să apelăm file.close()
```

Combinarea excepțiilor cu with

Exemplu complet:

```
try:
    with open("fisier.txt", "r") as file:
        data = file.read()
        print(data)
except FileNotFoundError:
    print("Fișierul nu există!")
```

De ce este util?

- Codul este mai clar și mai concis.
- Resursele sunt eliberate automat, chiar dacă apare o eroare.

Cuprins

1. Introducere
2. Structuri de control la nivel de instructiune
   - Secventa
   - Selectia
   - Repetitia
3. Subprograme
   - Efecte colaterale
   - Pseudonime
4. Exceptii
   - Exceptii in Ada
   - Exceptii in C#
   - Exceptii in Python
5. Pachete
   - Pachete in Java
   - Namespace-uri in C Sharp
6. Module
   - Module Java

Introducere în pachetele Java

Ce este un pachet în Java?

- Un pachet este un grup de clase și interfețe conexe.
- Oferă organizare și structură codului.
- Exemple: `java.util`, `java.io`.

Beneficiile utilizării pachetelor

- Organizare mai bună a codului.
- Evitarea conflictelor de nume.
- Permite controlul accesului (`public`, `protected`, `private`).
- Promovează reutilizarea codului.

Crearea unui pachet

Pasul 1: Definirea unui pachet

- Se utilizează cuvântul cheie package.

Exemplu:

```
package com.exemplu.util;

public class Salut {
    public void spuneSalut() {
        System.out.println("Salut!");
    }
}
```

Utilizarea unui pachet

Importul unui pachet:

- Se utilizează cuvântul cheie `import`.

Exemplu:

```
import com.exemplu.util.Salut;

public class Test {
    public static void main(String[] args) {
        Salut salut = new Salut();
        salut.spuneSalut();
    }
}
```

Tipuri de pachete Java

- **Pachete implicite:** Toate clasele fără o declarație `package`.
- **Pachete personalizate:** Create de programator.
- **Pachete standard Java:**
  - `java.lang` – Clase de bază.
  - `java.util` – Colecții, utilitare.
  - `java.io` – Operarea cu fișiere.

Pachetul java.lang

- Este importat implicit în toate programele Java.
- Conține clase fundamentale precum:
  - String
  - Math
  - Object

Exemplu:

```
String nume = "Java";
int lungime = nume.length();
System.out.println("Lungime: " + lungime);
```

Pachetul java.util

- Conține clase pentru colecții, data și ora, utilitare.
- Clase importante:
  - ArrayList
  - HashMap
  - Calendar

Exemplu:

```
import java.util.ArrayList;

ArrayList<String> lista = new ArrayList<>();
lista.add("Element");
System.out.println(lista);
```

Controlul accesului în pachete

Modificatori de acces:

- `public`: Accesibil din orice pachet.
- `protected`: Accesibil din pachet sau clase derivate.
- `default` (fără modificator): Accesibil doar în același pachet.
- `private`: Accesibil doar în aceeași clasă.

Sub-pachete

- Pachetele pot avea sub-pachete pentru organizare mai bună.
- Ex.: `com.exemplu.util` și `com.exemplu.servicii`.
- Clasele dintr-un sub-pachet trebuie importate separat.

Exemplu:

```
package com.exemplu.util;

public class Calculator {
    public int aduna(int a, int b) {
        return a + b;
    }
}
```

Recomandări pentru utilizarea pachetelor

- Folosește convenția de denumire bazată pe domeniul inversat (ex. `com.numecompanie.proiect`).
- Organizează logic clasele în pachete.
- Documentează clar funcționalitatea fiecărui pachet.
- Evită conflictele de nume utilizând pachete bine definite.

Concluzie

- Pachetele Java sunt esențiale pentru organizarea și reutilizarea codului.
- Oferă control al accesului și previn conflictele de nume.
- Utilizează pachetele Java standard pentru funcționalități comune.

Introducere în namespace-uri C#

Ce este un namespace?

- Un namespace este un container logic pentru clase, structuri, interfețe și alte elemente.
- Ajută la organizarea codului și evitarea conflictelor de nume.

Exemplu comun: `System`, `System.Collections.Generic`.

Beneficii ale utilizării namespace-urilor

- **Organizarea codului:** Gruparea logică a claselor.
- **Evitarea conflictelor:** Clase cu același nume pot coexista în namespace-uri diferite.
- **Claritate:** Îmbunătățirea lizibilității codului.
- **Control al accesului:** Organizare pe module sau biblioteci.

Crearea unui namespace

Cum definim un namespace?

```
namespace MyApplication {
    class Program {
        static void Main(string[] args) {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

- Namespace-ul `MyApplication` conține clasa `Program`.

Utilizarea namespace-urilor

Importul unui namespace:

```
using System;
using System.Collections.Generic;

namespace MyApp {
    class Program {
        static void Main(string[] args) {
            List<string> items = new List<string>();
        }
    }
}
```

- Cuvântul cheie using simplifică accesul la clase.

Namespace-uri implicite

- Fiecare proiect C# definește implicit un namespace bazat pe numele proiectului.
- Exemplu: Proiectul `MyApp` creează automat namespace-ul `MyApp`.
- Se pot adăuga namespace-uri suplimentare în cadrul proiectului.

Namespace-uri și clase cu același nume

Gestionarea conflictelor:

- Clase cu același nume pot fi diferențiate prin namespace.

Exemplu:

```
namespace A {
    class MyClass { }
}

namespace B {
    class MyClass { }
}

class Program {
    static void Main(string[] args) {
        A.MyClass objA = new A.MyClass();
        B.MyClass objB = new B.MyClass();
    }
```

Namespace-uri cu aliasuri

Folosirea aliasurilor:

```
using ProjA = ProjectA.Namespace1;
using ProjB = ProjectB.Namespace2;

class Program {
    static void Main(string[] args) {
        ProjA.Class1 objA = new ProjA.Class1();
        ProjB.Class2 objB = new ProjB.Class2();
    }
}
```

- Aliasurile reduc complexitatea codului.

Namespace-uri în structura proiectului

- Structura fișierelor proiectului influențează namespace-urile.
- Fiecare fișier poate avea propriul namespace.

Exemplu de structură:

```
Project/
|-- Models/
|    |-- User.cs (namespace Project.Models)
|-- Controllers/
     |-- UserController.cs (namespace Project.Controllers)
```

Namespace-uri standard în .NET

- `System`: Clase de bază (ex. `Console`, `Math`).
- `System.Collections.Generic`: Colecții generice (ex. `List`, `Dictionary`).
- `System.Linq`: Funcționalități LINQ.
- `System.IO`: Manipularea fișierelor și fluxurilor.

Exemplu:

```
using System.IO;

File.WriteAllText("test.txt", "Hello, World!");
```

Recomandări pentru namespace-uri

- Urmează convențiile de denumire bazate pe structura proiectului.
- Evită namespace-uri prea lungi sau complexe.
- Utilizează namespace-uri pentru a organiza logic funcționalitățile.
- Documentează relațiile dintre namespace-uri în proiecte mari.

Cuprins

1. Introducere
2. Structuri de control la nivel de instructiune
   - Secventa
   - Selectia
   - Repetitia
3. Subprograme
   - Efecte colaterale
   - Pseudonime
4. Exceptii
   - Exceptii in Ada
   - Exceptii in C#
   - Exceptii in Python
5. Pachete
   - Pachete in Java
   - Namespace-uri in C Sharp
6. Module
   - Module Java

Module reutilizabile în Java

Ce sunt modulele reutilizabile?

- Modulele sunt componente independente care pot fi utilizate în mai multe aplicații.
- Permit dezvoltarea modulară a aplicațiilor, reducând redundanța și facilitând întreținerea.

Avantajele modulelor reutilizabile

- **Reducerea codului duplicat:** Se reutilizează codul în loc să fie recreat.
- **Ușurința întreținerii:** Orice modificare într-un modul afectează toate utilizările.
- **Separarea responsabilităților:** Codul este împărțit logic în funcționalități specifice.

Structura unui modul în Java

- Un modul Java este definit într-un fișier `module-info.java`.
- Exemple de directive:
  - `requires`: Specifică dependențele modulului.
  - `exports`: Face pachetele accesibile altor module.

Exemplu:

```
module com.example.mylibrary
{
    exports com.example.utils;
    requires java.sql;
}
```

Exemplu de modul reutilizabil

Un modul pentru validarea intrărilor utilizatorului

- Funcționalitate: Validarea adreselor de e-mail și a numerelor de telefon.
- Pachet: `com.example.validation`

```
package com.example.validation;

public class Validator
{
    public static boolean isValidEmail(String email)
{
        return email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");
    }

    public static boolean isValidPhone(String phone) {
        return phone.matches("^\\+?[0-9]{10,13}$");
    }
}
```

Utilizarea unui modul reutilizabil

Exemplu: Validarea în cadrul unei aplicații.

```
import com.example.validation.Validator;

public class Main {
    public static void main(String[] args) {
        String email = "test@example.com";
        String phone = "+1234567890";

        if (Validator.isValidEmail(email)) {
            System.out.println("Email valid.");
        } else {
            System.out.println("Email invalid.");
        }

        if (Validator.isValidPhone(phone)) {
            System.out.println("Număr valid.");
        } else {
            System.out.println("Număr invalid.");
        }
    }
}
```

Separarea codului

- **Definirea funcționalităților distincte:** Crearea de module pentru fiecare responsabilitate.
- Exemple:
  - Modul pentru accesul la bazele de date.
  - Modul pentru interfața grafică.
  - Modul pentru logare și raportare.
- Asigură scalabilitatea și testabilitatea codului.

Ce sunt ansamblele în C#?

- **Definiție:** Ansamblul (*assembly*) este o unitate de compilare și distribuire în .NET.
- **Tipuri:**
  - Biblioteci de clase (`.dll`).
  - Aplicații executabile (`.exe`).
- **Rol:** Reutilizarea codului în diverse aplicații prin ansamble externe.

Avantajele ansamblelor reutilizabile

- **Reutilizare:** Codul scris o dată poate fi folosit în mai multe proiecte.
- **Modularitate:** Codul este organizat în module independente.
- **Întreținere:** Actualizările într-un ansamblu se propagă automat în toate aplicațiile.
- **Distribuire ușoară:** Pot fi distribuite separat ca fișiere `.dll`.

Crearea unui ansamblu reutilizabil

- **Pasul 1:** Crearea unui proiect de tip `Class Library` în Visual Studio.
- **Pasul 2:** Adăugarea funcționalităților dorite.
- **Pasul 3:** Construirea proiectului pentru a genera fișierul `.dll`.

```
// Fișier: MathUtilities.cs
namespace Utilities {
    public class MathUtilities {
        public static int Add(int a, int b) {
            return a + b;
        }
    }
}
```

Utilizarea unui ansamblu reutilizabil

- **Pasul 1:** Referențierea ansamblului `.dll` în proiectul curent.
- **Pasul 2:** Importul spațiului de nume (`namespace`).
- **Pasul 3:** Apelarea metodelor definite în ansamblu.

```
// Fișier: Program.cs
using Utilities;

class Program {
    static void Main() {
        int result = MathUtilities.Add(5, 3);
        Console.WriteLine($"Rezultatul: {result}");
    }
}
```

Organizarea ansamblelor

- **Gruparea logică:** Clasele și metodele similare ar trebui să fie grupate în același ansamblu.
- **Minimizarea dependențelor:** Evitați dependențele inutile între ansamble.
- **Versionare:** Utilizați versiuni clare pentru ansamble pentru a facilita actualizările.
- **Documentare:** Fiecare ansamblu ar trebui să includă documentație pentru utilizatori.

Exemple de ansamble comune în .NET

- **System.dll:** Funcționalități de bază, cum ar fi gestionarea stringurilor, colecțiilor etc.
- **System.IO.dll:** Funcții pentru accesarea sistemului de fișiere.
- **System.Data.dll:** Funcții pentru interacțiunea cu baze de date.
- **Ansambluri particularizate:** Creat de dezvoltatori pentru funcții specifice.

Depanarea ansamblelor reutilizabile

- **Erori comune:**
  - `FileNotFoundException`: Ansamblul lipsă.
  - `TypeLoadException`: Tipuri incompatibile între versiunile ansamblului.
- **Soluții:**
  - Asigurați-vă că fișierul `.dll` este inclus în referințe.
  - Folosiți versiuni compatibile ale ansamblului.

Testarea ansamblelor reutilizabile

- **Unit Testing:** Utilizați framework-uri precum `xUnit` sau `NUnit`.
- **Exemplu de test:**

```
using Utilities;
using Xunit;

public class MathUtilitiesTests {
    [Fact]
    public void TestAdd() {
        int result = MathUtilities.Add(2, 3);
        Assert.Equal(5, result);
    }
}
```

Cuprins

1. Introducere
2. Structuri de control la nivel de instructiune
   - Secventa
   - Selectia
   - Repetitia
3. Subprograme
   - Efecte colaterale
   - Pseudonime
4. Exceptii
   - Exceptii in Ada
   - Exceptii in C#
   - Exceptii in Python
5. Pachete
   - Pachete in Java
   - Namespace-uri in C Sharp
6. Module
   - Module Java

Bibliografie

1. Horia Ciocarlie – The programming language universe, second edition, Timisoara, 2013.
2. Carlo Ghezzi, Mehdi Jarayeri – Programming Languages, John Wiley, 1987.
3. Ellis Horrowitz – Fundamentals of programming languages, Computer Science Press, 1984.
4. Donald Knuth – The art of computer programming, 2002.